

Programación Avanzada
Grado en Ingeniería Informática - Curso 2026/2027

EPD 4: JavaScript Avanzado

IMPORTANTE: Las entregas de este guion deben respetar lo establecido en el documento "Instrucciones de entregas de EPD", suponiendo en caso contrario penalizaciones en las calificaciones obtenidas. Consulte dicho documento antes de realizar una entrega.

Objetivos

- Tratar eventos en JavaScript.
- Manejar objetos propios de JavaScript.

Resultados de aprendizaje

1. Identifica y aplica eventos en JavaScript para permitir la interacción dinámica entre el usuario y la aplicación.
2. Utiliza *event* para gestionar correctamente las acciones del ratón y del teclado.
3. Implementa validaciones de formularios a través del uso de eventos, garantizando la validez de los datos ingresados.
4. Emplea funciones de temporización (*setTimeout*, *clearTimeout*, *setInterval*, *clearInterval*) para controlar acciones programadas o repetitivas en la interfaz.
5. Comprende y aplica el modelo de propagación de eventos para gestionar adecuadamente las interacciones entre elementos del DOM.
6. Manipula el DOM para acceder, modificar o crear nodos, permitiendo actualizar el contenido y estructura de la página de forma dinámica.
7. Aplica estilos y modificaciones visuales a los elementos del DOM mediante JavaScript para mejorar la presentación e interactividad.
8. Integra objetos del navegador (como *window*, *document* y *location*) para desarrollar interfaces interactivas y funcionales.

Conceptos

1. Eventos

JavaScript aplica el modelo de programación basada en eventos. En este tipo de paradigma de programación, los *scripts* esperan la interacción del usuario, o de cualquier otro actor, para comenzar el procesamiento y generar un resultado.

La interacción que realiza el usuario se transmite a los programas mediante una serie de llamadas que denominamos genéricamente como *eventos*. JavaScript define numerosos eventos que permiten una interacción completa entre el usuario y las páginas/aplicaciones web. Por ejemplo, la pulsación de una tecla constituye un evento (en realidad, son una serie de eventos que constituyen la pulsación de la tecla, soltar la tecla, etc.), así como pinchar o mover el ratón, seleccionar un elemento de un formulario, redimensionar la ventana del navegador, etc. Igualmente, otros actores, como el navegador, pueden provocar eventos. Un ejemplo de este último tipo es el evento que se da una vez cargada la página.

JavaScript permite asignar un código a cada uno de los eventos. De esta forma, cuando se produce un evento de un determinado tipo, JavaScript ejecuta dicho código. Como buena práctica, el código asociado a un evento se limita, prácticamente, a invocar una función, que es el lugar donde realmente realizaremos el procesamiento necesario para tratar el evento, consiguiendo así un código más legible. Este tipo de funciones se denominan "*event handlers*" o en castellano "manejadores de eventos".

Puede encontrar más información en [1], capítulo 32, pág. 1043-1097 y en [2] capítulo 6 pág. 63-80.

2. Objetos

JavaScript es un lenguaje interpretado de secuencias de comandos (*scripts*) orientado a prototipos. La necesidad de un intérprete para ejecutarse (habitualmente en un navegador web, aunque también hay usos del mismo en servidor) limita su funcionalidad. Aunque tiene menos capacidades que los lenguajes de propósito general, como por ejemplo C++ o Java,

JavaScript ofrece unas características básicas asimilables a las de un lenguaje *orientado a objetos*, pero carece de *clases*. De esta forma, se puede decir que JavaScript es un lenguaje *orientado a objetos basado en prototipos* (ya que suple la carencia de clases por un mecanismo de clonación de objetos).

En JavaScript, los objetos pueden ser vistos como colecciones de propiedades y métodos. Los objetos en este lenguaje se pueden categorizar de la siguiente forma:

1. Objetos intrínsecos (Array, Boolean, Date, Function, Math, Number, Object y String)[1].
2. Objetos del navegador (window, document, form, frame, history, location).
3. Objetos creados por el usuario.

Las primeras dos categorías están constituidas por objetos que forman parte de la implementación del lenguaje, es decir, ya están construidos y disponibles para que el programador los utilice. La tercera categoría la conforman los objetos creados por el programador, para dar solución a sus necesidades.

Puede encontrar más información sobre el modelo de objetos en JavaScript en [1] Parte IV pág. 501-1043 y capítulo 6 pág. 79-82

Bibliografía Básica

1. JavaScript Bible. Seventh Edition. Danny Goodman, Michael Morrison et al. Wiley Publishing, 2010. Parte IV https://ebookcentral.proquest.com/lib/bupo-ebooks/reader.action?docID=624389&query=JavaScript

Experimentos

E1. (30 mins.) Resultados de aprendizaje: 2, 4, 5, 6 y 8

En el presente experimento estudiaremos algunos ejemplos de código JavaScript para el manejo de eventos. Para ello, emplearemos un *script* cuya función es realizar una cuenta atrás. El usuario ha de introducir los valores de las horas, minutos y segundos correctamente para poder empezar dicha cuenta. Por tanto, podemos dividir este experimento en dos partes:

a) Validación de los campos a introducir. Dichos campos han de ser validados de modo que sólo puedan introducirse valores numéricos y, en caso de que el número de segundos o minutos sea mayor de 60, se actualicen los campos correspondientes. Las funciones del experimento correspondiente con esta parte son:

> *esNumero(e)*: dado el evento *e*, comprueba si la tecla presionada es o no un número. En ese caso devuelve un valor *true*, y *false* en caso contrario.
> *actualizaHoras( )*: evita que el campo correspondiente a las horas esté vacío (cadena vacía).
> *actualizaMinutos( )*: comprueba que el rango introducido sea correcto y, en caso contrario (> 59), actualiza el campo de las horas.
> *actualizaSegundos( )*: comprueba que el rango introducido sea correcto y, en caso contrario (> 59), actualiza los campos correspondientes (minutos y horas).

b) Cronómetro. En este caso se hace uso de las funciones *setTimeout* para actualizar la cuenta y *clearTimeout* para cancelar la cuenta iniciada con la función anterior. Las funciones que corresponden al cronómetro son:

> *detenerCrono( )*: para el cronómetro llamando a la función de JavaScript *clearTimeout*.
> *iniciarCrono( )*: comienza la cuenta atrás y llama a *actualizaCrono*.
> *actualizaCrono( )*: función que actualiza el cronómetro. Para ello, usa la función de JavaScript *setTimeout* para que ejecute *actualizaCrono* una décima de segundo más tarde. De esta forma, el cronómetro se irá actualizando diez veces por segundo.

Encontrará el código del experimento en el material adicional a este guion.

[1] Obsérvese que el lenguaje es sensible al caso y que el nombre de este tipo de objetos comienza por mayúscula.

E2. (30 mins.) Resultados de aprendizaje: 1, 2, 3, 5 y 8

Cree un formulario de *login* que impida enviar los datos al servidor hasta que ambos campos (usuario y contraseña) se encuentren rellenos ¿Puede reutilizar la misma función para comprobar ambos campos o necesita hacer una función para cada uno de ellos? ¿Qué es fundamental para poder reutilizar las funciones de comprobación? ¿Cómo se comporta la propagación de eventos en este caso? En base a lo anterior ¿Cómo podemos conseguir que el formulario no se envíe si no se cumplen las restricciones establecidas? ¿Por qué se debe procesar ese tipo de restricciones en el cliente, qué ventajas tiene? ¿Es suficiente con implementar estas restricciones en JavaScript o se debe hacer algo en el lado del servidor?

Ejercicios

CONSIDERACIONES:

- **El guión trata sobre la creación de funcionalidades a través de JS para el manejo avanzado de esta tecnología. En esta práctica únicamente se valorarán las competencias relacionadas con JS.**
- **Para la evaluación de esta práctica se escogerá un ejercicio al azar para ser evaluado. La nota total de la práctica dependerá de dicha calificación. Entregue cada ejercicio y problema en una carpeta diferente antes de subir el archivo zip como entrega.**
- **En los siguientes ejercicios, además de la calidad técnica se valorará la limpieza de código y la optimización del mismo.**
- **Por cada ejercicio, incluya únicamente los archivos necesarios para resolver el ejercicio. Si usa un IDE, no envíe todo el proyecto por cada ejercicio.**

Ej1. (60 mins.) Resultados de aprendizaje: 1, 3, 4, 5, 6, 7 y 8

Desarrolle una aplicación web interactiva que simule el panel de control de una misión espacial. El usuario asumirá la dirección del vuelo y deberá preparar la nave, activar sus cinco sistemas principales en una secuencia estricta y autorizar el despegue antes de que concluya la cuenta atrás.

Toda la interacción se realizará mediante botones, eventos de teclado, eventos de ratón y elementos del DOM creados y actualizados dinámicamente usando JavaScript.

Estructura inicial de elementos HTML en la interfaz

La página deberá mostrar los siguientes elementos:

- Nombre de la misión. Por ejemplo: `Misión Apolo`.
- Estado general de la nave.
- Contador de tiempo: Temporizador regresivo.
- Panel con los sistemas principales de la nave: Contenedor de información que alberga la información de los diferentes sistemas de la nave.
- Tarjetas de sistemas de la nave: Elementos independientes para cada sistema de la nave.
- Consola de mensajes: Área de registro de mensajes que listará cronológicamente todos los eventos y acciones que transcurren en la misión.
- Panel lateral de información: Zona del panel orientada a mostrar la descripción operativa del sistema individual seleccionado.
- Botón `Iniciar misión`.
- Botón `Lanzar nave`.
- Botón `Abandonar misión`.

Los sistemas principales que deben aparecer en el panel son:

- `Energía`.
- `Comunicaciones`.
- `Navegación`.
- `Soporte vital`.
- `Propulsión`.

Cada sistema se representará mediante un bloque independiente. Para ello use elementos de tipo `<div>`. Cada tarjeta de sistema mostrará el nombre del propio sistema, su estado actual (`desactivado` u `operativo`) y un botón de activación individual.

Inicio de la misión

Al pulsar el botón `Iniciar misión`, la aplicación deberá:

- Se deshabilitará y se ocultará el botón de inicio.
- Se habilitará el botón `Abandonar misión.`
- Se habilitarán los controles de activación de los sistemas.
- Cambiará el estado general de la nave a: `Preparación en curso.`
- Se registrará en la consola el mensaje: `Protocolo de preparación iniciado.`
- Se activará la cuenta atrás de 90 segundos. La cuenta atrás deberá actualizarse cada segundo mediante la función `setInterval()`. Cuando queden 15 segundos, el contador deberá resaltarse visualmente usando una clase CSS con color rojo y parpadeo. Además, deberá mostrar una advertencia en la consola de mensajes: `ADVERTENCIA: Tiempo de preparación próximo a finalizar.`
- En el caso de que el contador alcance los 0 segundos sin haberse lanzado la nave:
    - Se detendrá el temporizador principal. Use para ello la función `clearInterval().`
    - Se bloquearán todos los botones y la interacción por teclado.
    - El estado de la nave cambiará a `MISIÓN CANCELADA: Protocolo incompleto.`
    - Se mostrará en el área principal un resumen con los sistemas completados y aquellos que están pendientes:

```
MISIÓN CANCELADA
Sistemas operativos: [X] de 5
Sistemas pendientes: [Lista de sistemas no activados]
Estado: Protocolo incompleto
Resultado: Lanzamiento no autorizado
```

Activación de sistemas

Los sistemas deben iniciarse de manera secuencial y respetando obligatoriamente un orden estricto: `Energía`, `Comunicaciones`, `Navegación`, `Soporte vital` y `Propulsión`.

Cuando se realice una activación y sea correcta:

- El sistema pasa a estado `Operativo.`
- Se añade una clase CSS correspondiente a la tarjeta para modificar su aspecto visual. Por ejemplo: borde verde o fondo iluminado.
- El botón de activación de dicha tarjeta queda deshabilitado para impedir futuras reactivaciones.
- Se actualiza el contador visible de sistemas operativos. Por ejemplo: `Sistemas operativos: 3 de 5.`
- Se añade un registro de texto a la consola indicando el sistema y el método de interacción utilizado. Esta parte será descrita en el apartado de eventos.

En cambio, si el usuario intenta activar un sistema antes de haber activado el sistema inmediatamente anterior (Por ejemplo, si intenta activar la `Navegación` sin haber activado el módulo de `Comunicaciones`), la acción quedará bloqueada:

- Se mostrará un `popup` de información con el mensaje: `Acción bloqueada: debe activar primero el sistema anterior.`
- El mensaje desaparecerá automáticamente tras 3 segundos mediante `setTimeout()`. Dicho mensaje debe gestionarse mediante un elemento HTML (Por ejemplo, puede usar la etiqueta `<div>`) sobre la propia interfaz. Evite usar `window.alert()` ya que no se puede cerrar desde JavaScript.
- Para controlar el solapamiento de mensajes, si se produce un nuevo intento erróneo mientras hay un aviso visible, debe cancelarse el temporizador anterior con `clearTimeout()` antes de lanzar el nuevo evento. Ésto evitará que el mensaje desaparezca prematuramente.

Interacción con el teclado

Además de los botones, los sistemas podrán activarse usando las teclas numéricas:

- Tecla `1`: `Energía`.
- Tecla `2`: `Comunicaciones`.
- Tecla `3`: `Navegación`.
- Tecla `4`: `Soporte vital`.
- Tecla `5`: `Propulsión`.

La aplicación deberá utilizar el objeto *`event`* junto con el evento `keydown` para identificar y capturar la tecla pulsada. Para ello, use `event.key`. Las pulsaciones serán válidas cuando la misión haya iniciado y el sistema correspondiente no haya sido activado previamente. También se deberá registrar en la consola si la activación se ha realizado mediante teclado o mediante ratón.

Ejemplos:

- Activación por botón usando el ratón: Al pulsar el botón de activación de una tarjeta válida, se registra en consola: `[Nombre del sistema] activado mediante ratón`.
- Activación por botón usando el teclado: Al pulsar una tecla válida en la secuencia correcta, se registrará en consola: `[Nombre del sistema] activado mediante teclado`.

Propagación de eventos y manejo del DOM

- Al hacer clic sobre cualquier punto de una tarjeta (fuera del botón de activación), se mostrará en el panel lateral la descripción operativa del sistema correspondiente. Por ejemplo: *`Navegación: Controla la trayectoria, posición y orientación de la nave`*.
- El clic sobre el botón de activación del sistema no debe desencadenar la apertura de la descripción de la tarjeta. Debe emplearse `event.stopPropagation()` en el evento del botón para evitar el efecto indeseado.
- Al hacer clic directamente sobre el contenedor principal, en su espacio o márgenes exteriores y sin haber pinchado en ninguna tarjeta, se registrará en la consola: `Panel de sistemas seleccionado`. Para ello, a nivel técnico, verifique que `event.target` coincide con `event.currentTarget` o asegure la detención del evento en las tarjetas.
- Implemente eventos para asociar y retirar una clase visual de resaltado temporal al situar el puntero sobre cada tarjeta. Para ello, ayúdese de `mouseover` y `mouseout`.
- Al hacer doble clic sobre la tarjeta de un sistema, la aplicación desplegará un cuadro de diálogo nativo del navegador (`window.alert()`) con las especificaciones de dicho sistema. Por ejemplo, *`ESPECIFICACIONES DE ENERGÍA: Generador por fisión de 250 kW, respaldo auxiliar por celdas fotovoltaicas`*.

Secuencia de lanzamiento

- El botón `Lanzar nave` permanecerá deshabilitado mientras falte cualquier sistema por activar.
- En cuanto los cinco sistemas alcancen el estado `Operativo`:
  - El botón `Lanzar nave` se habilitará.
  - El estado general cambiará a: `Nave preparada para el lanzamiento`.
  - Se bloqueará la modificación de los sistemas.
- Al pulsar `Lanzar nave`:
  - Se detendrá de inmediato la cuenta atrás de 90 segundos mediante `clearInterval()`, congelando los segundos restantes para el balance final.
  - Se inhabilitarán todos los botones de la interfaz.
  - Se iniciará una cuenta atrás de lanzamiento de 5 segundos con intervalos de 1 segundo mediante `setInterval()`:
    ```
    5 -> 4 -> 3 -> 2 -> 1 -> ¡Despegue!
    ```
  - Concluido el despegue, se modificará el título de la pestaña del navegador mediante `document.title` y se presentará el informe final de lanzamiento:

    ```
    Título: "¡Misión en Vuelo! - Lanzamiento Exitoso"

    ¡LANZAMIENTO COMPLETADO!
    ```

```
Misión: [Nombre de la misión]
Sistemas operativos: 5 de 5
Tiempo restante: [Segundos restantes congelados] segundos
Estado: Nave en vuelo

Resultado: Protocolo ejecutado correctamente
```

Abandono de la misión

Al pulsar el botón `Abandonar misión`:

- Se solicitará confirmación al usuario mediante `window.confirm("¿Confirma que desea abortar la misión en curso?")`.
- En caso afirmativo:
  - Se cancelarán todos los temporizadores activos. Use las funciones `clearInterval()` y `clearTimeout()`.
  - Se deshabilitarán todos los controles y uso del teclado.
  - Se actualizará el estado general a: `Misión abortada por el usuario`.
  - Se registrará en la consola: `Aborto de misión confirmado`.
  - Se cambiará el título de la página a `Misión abortada`.

Eventos y funciones requeridas

La solución deberá utilizar, como mínimo, los siguientes eventos:

- *`click()`*, para activar sistemas y manejar los botones.
- *`keydown()`*, para controlar los sistemas mediante el teclado.
- *`mouseover()`*, para resaltar temporalmente una tarjeta.
- *`mouseout()`*, para retirar el resaltado.
- *`dblclick()`*, para mostrar información técnica ampliada de un sistema.

Ejemplo de interacción correcta

```
El usuario inicia la misión.
Se activa el contador de 90 segundos.
Pulsa la tecla 1.
El sistema de energía cambia a "Operativo".
Se realiza un clic en el botón de comunicaciones.
La consola registra el método utilizado en cada activación.
Intenta activar la propulsión antes de la navegación.
Aparece una advertencia durante tres segundos.
Completa correctamente los cinco sistemas.
Se habilita el botón "Lanzar nave".
Se ejecuta la cuenta atrás final.
La aplicación muestra el resultado del lanzamiento.
```

Ejemplo de interacción incorrecta

```
El usuario inicia la misión y activa únicamente los sistemas de energía y
comunicaciones.
Cuando quedan 15 segundos: "ADVERTENCIA: tiempo de preparación próximo a finalizar".
El usuario intenta activar soporte vital sin haber activado navegación.
La aplicación muestra: "Acción bloqueada: debe activar primero el sistema anterior".
El tiempo llega a cero y se obtiene el siguiente resultado: MISIÓN CANCELADA
Sistemas operativos: 2 de 5
Sistemas pendientes: Navegación, soporte vital y propulsión
Estado: Protocolo incompleto
Resultado: Lanzamiento no autorizado
```

Requisitos técnicos adicionales

- No se permitirá escribir directamente los mensajes de estado final en el código HTML.
- Los estados deberán gestionarse mediante variables, vectores u objetos de JavaScript.

- La aplicación deberá incluir un mecanismo para reiniciar la misión (por ejemplo, un botón *Reiniciar misión* o *Nueva misión* que se habilite al finalizar, cancelarse o al abandonar la misma) que restablezca todas las variables, temporizadores y elementos del DOM a su estado inicial. No se considerará válida una solución que obligue al usuario a recargar la página en el navegador (*F5*) para volver a iniciar una misión.
- Una misma acción no podrá ejecutarse más de una vez.
- Los temporizadores deberán detenerse correctamente al lanzar, cancelar o finalizar la misión.
- Se valorará la claridad de la interfaz, la organización del código y la correcta separación entre estructura HTML, estilos CSS y lógica JavaScript.

Ej2. (60 mins.) resultados de aprendizaje: 1, 4, 5, 6 y 8

Implemente una aplicación web interactiva que simule la consola de un Centro de Operaciones de Ciberseguridad (SOC). El usuario se convertirá en un analista y deberá resolver un turno de cinco incidencias que amenazan la infraestructura informática de la universidad.

Cada incidencia dispone de un límite de tiempo para ser diagnosticada y resuelta. La aplicación evaluará la toma de decisiones, bonificará las respuestas rápidas, penalizará los fallos y generará un informe técnico final acreditativo del desempeño.

Incidencias

Las incidencias se almacenarán en un vector bidimensional en el que cada elemento es un vector con la siguiente estructura: `[categoría, descripción, gravedad, opciones[], indiceCorrecto, explicacion]`

La aplicación contará con un banco de datos que debe cubrir las seis categorías fijadas por la normativa de seguridad de la universidad:

```
const incidencias = [
  [
    "Seguridad",
      "Se detectan numerosos intentos fallidos de acceso por fuerza bruta desde una
dirección IP desconocida.",
    "Alta",
    [
      "Reiniciar el servidor web",
      "Bloquear temporalmente la dirección IP en el firewall",
      "Eliminar todas las cuentas de usuario afectadas",
      "Desactivar la infraestructura de red completa"
    ],
    1,
     "El bloqueo perimetral en el firewall frena el ataque de inmediato sin provocar
denegación de servicio a los usuarios legítimos."
  ],
  [
    "Copias de seguridad",
     "El proceso programado de respaldo nocturno de las bases de datos académicas ha
finalizado con estado erróneo.",
    "Media",
    [
      "Ignorar la alerta y esperar a la noche siguiente",
      "Formatear la cabina de almacenamiento de copias",
      "Examinar los logs de error y lanzar un respaldo manual inmediato",
      "Eliminar las copias históricas para liberar espacio"
    ],
    2,
    "Analizar el registro permite identificar el motivo del fallo y el respaldo manual
garantiza contar con un punto de recuperación reciente."
  ],
  [
    "Redes",
    "Los puestos de trabajo de un departamento no tienen conexión a Internet, aunque
sí se comunican con los servidores locales.",
    "Media",
    [
      "Verificar la puerta de enlace predeterminada y el router de salida",
      "Sustituir los cables de red de todos los ordenadores",
      "Reinstalar el sistema operativo de las máquinas cliente",
```

```
      "Apagar el conmutador (switch) de planta"
    ],
    0,
       "La comunicación local operativa confirma que el fallo se localiza en el
enrutamiento o en el enlace de salida hacia el proveedor externo."
  ],
  [
    "Hardware",
      "El sensor térmico del servidor de correo alerta de una temperatura crítica
sostenida en la CPU superior a 95°C.",
    "Alta",
    [
      "Aumentar la carga de trabajo para forzar la ventilación",
      "Apagar ordenadamente la máquina y verificar disipación y flujo de aire",
      "Desactivar las alertas de temperatura de la BIOS",
      "Desconectar el cable de alimentación de forma abrupta"
    ],
    1,
     "El apagado ordenado evita daños físicos en el silicio y permite inspeccionar los
ventiladores y la pasta térmica sin corromper datos."
  ],
  [
    "Aplicaciones",
      "El portal de matriculación arroja un error '500 Internal Server Error' tras
desplegar un parche de actualización.",
    "Alta",
    [
      "Solicitar a los alumnos que insistan recargando la página",
      "Revertir el despliegue a la versión estable previa (rollback)",
      "Borrar la base de datos de expedientes",
      "Cambiar el nombre del dominio institucional"
    ],
    1,
     "Efectuar un rollback inmediato restituye el servicio a la comunidad universitaria
mientras se depura el parche en entorno de pruebas."
  ],
  [
    "Gestión de usuarios",
       "Un profesor comunica que ha recibido un correo solicitando sus claves
universitarias y sospecha haber caído en una trampa de phishing.",
    "Alta",
    [
      "Restablecer su contraseña inmediatamente y revocar sesiones activas",
      "Publicar un aviso en redes sociales advirtiendo del hecho",
      "Ignorar el aviso si el profesor aún puede iniciar sesión",
      "Dar de baja la matrícula de todos sus alumnos"
    ],
    0,
     "Inhabilitar la contraseña vulnerada e invalidar los tokens de sesión impide el
acceso del atacante a los sistemas de la universidad."
  ]
];
```

Interfaz de la aplicación

La pantalla de inicio mostrará:

- Título de la aplicación: `Centro de Operaciones de Ciberseguridad`.
- Descripción del puesto y objetivos de la guardia.
- Botón `Iniciar turno`.
- Marcadores de estadísticas en blanco (Puntuación: 0, Incidencias resueltas: 0/5).
- Una barra de progreso al 0%.
- Historial de actividad vacío.

Mecánica de turnos

- Inicio del turno:
    - Se reinicia el estado de la partida: Cambie la puntuación, el contador de aciertos, los fallos y las omisiones a 0.

    - Se vacía el contenedor del historial de actividad.
    - Se seleccionan aleatoriamente cinco incidencias del banco de preguntas, asegurando que no se repita ninguna dentro del mismo turno.
    - Se oculta el botón de inicio.
    - Se carga la primera incidencia y se arranca su temporizador.

- Temporización por pregunta. Para cada una de las 5 incidencias, el analista dispondrá de un límite de 25 segundos:
    - El contador visible descenderá de 25 a 0 cada segundo mediante `setInterval()`.
    - Se programará un cierre automático a los 25 segundos mediante `setTimeout()`.
    - Cuando el contador alcance los 5 segundos restantes:
        - El contador cambiará su estilo visual (color rojo y resaltado parpadeante).
        - Se mostrará el aviso: `Atención: poco tiempo de respuesta`.

Opciones

- Las cuatro posibles actuaciones se generarán en el DOM como botones interactivos creados dinámicamente.
- Interacción por teclado:
    - Las teclas `1`, `2`, `3` y `4` del teclado numérico seleccionarán directamente las respuestas 1ª, 2ª, 3ª y 4ª respectivamente. Haga uso del objeto `event.key`.
    - Maneje los eventos `mouseover` y `mouseout` sobre los botones para aplicar y retirar una clase visual.

Sistema de puntuación

- Respuesta correcta: +15 puntos.
- Bonificación por rapidez: Si la respuesta correcta se selecciona durante los primeros 10 segundos desde la presentación de la incidencia, se sumará un extra de +5 puntos (total de +20 puntos en esa incidencia).
- Respuesta incorrecta: -5 puntos.
- Tiempo agotado: -3 puntos.
- La puntuación acumulada global nunca podrá ser inferior a 0. Si una penalización redujera el marcador por debajo de cero, la puntuación total se mantendrá en 0.

Evaluación y transición

Al hacer clic en una opción o pulsar su tecla equivalente, el sistema deberá:

- Cancelar los temporizadores de la incidencia. Use `clearInterval()` y `clearTimeout()`.
- Deshabilitar todos los botones y activar un bloqueo para impedir que nuevas pulsaciones de ratón o teclado sean registradas.
- Retirar cualquier clase de resaltado aplicada por mouseover y aplicar las clases visuales de resultado:
    - El botón elegido se resalta en verde (si es correcto) o rojo (si es incorrecto).
    - La opción correcta siempre se resalta en verde para retroalimentación pedagógica.
- Se muestra un panel explicativo con el veredicto, el tiempo empleado y la justificación técnica:

Ejemplo con bonificación:

```
DECISIÓN CORRECTA (+20 pts)
[Explicación de la solución almacenada en el vector]
Tiempo de respuesta: 7 segundos (¡Bonificación aplicada!)
```

Ejemplo incorrecto:

```
DECISIÓN INCORRECTA (-5 pts)
Actuación recomendada: [Texto de la opción correcta]
[Explicación de la solución]
Tiempo de respuesta: 14 segundos
```

- Se añade la entrada correspondiente al registro histórico de actividad.
- Se actualiza la barra de progreso (20% por cada incidencia completada).
- Transcurridos exactamente 2 segundos mediante un `setTimeout()`, se carga automáticamente la siguiente incidencia (o se finaliza el turno si era la quinta).

Tiempo agotado

Si transcurren los 25 segundos sin respuesta:

- Se para el contador.
- Se bloquean las interacciones de ratón y teclado.
- Se aplica la penalización de -3 puntos.
- Se resalta visualmente la opción correcta en color verde y se muestra el mensaje explicativo:

```
TIEMPO DE RESPUESTA AGOTADO (-3 pts)
No se ha aplicado ninguna actuación a tiempo.
Actuación recomendada: [Texto de la opción correcta]
[Explicación de la solución]
```

Además, se añade al registro histórico como incidencia desatendida y, tras 2 segundos, se pasa a la siguiente.

Registro de actividad

Cada incidencia completada generará dinámicamente una tarjeta o fila en el panel de actividad con el siguiente formato:

```
Incidencia [N] | Categoría: [Categoría] | Gravedad: [Nivel]
Decisión adoptada: [Texto de la opción o "Sin respuesta"]
Resultado: [Correcta / Incorrecta / Tiempo agotado]
Tiempo empleado: [S] segundos
Variación de puntuación: [+20 / +15 / -5 / -3]
```

Finalización del turno

Al concluir la 5ª incidencia:

- Se detienen todos los procesos.
- Se genera y muestra un informe técnico final detallado en el contenedor principal:

```
=====================================================
INFORME TÉCNICO DE FIN DE GUARDIA (SOC)
=====================================================
Incidencias analizadas: 5
Decisiones correctas: [Cantidad]
Decisiones incorrectas: [Cantidad]
Incidencias sin respuesta: [Cantidad]
Puntuación final obtenida: [Puntos]
Tiempo medio de respuesta: [Media] segundos
Nivel de desempeño asignado: [Clasificacion]
=====================================================
```

- Puntuaciones:
  - Mayor o igual a 80 puntos: Especialista.
  - Entre 55 y 79 puntos: Técnico avanzado.
  - Entre 30 y 54 puntos: Técnico en formación.
  - Menor a 30 puntos: Debe revisar el protocolo de actuación.
- Se habilitará un botón `Iniciar nuevo turno` que restablecerá por completo las variables y la interfaz sin recargar la página web.

Funciones recomendadas para realizar el problema

```
function iniciarTurno() {
  // Inicializa variables, baraja y selecciona 5 incidencias, arranca primera
}

function seleccionarIncidencias() {
  // Retorna un subconjunto aleatorio de 5 incidencias sin repetición
}

function mostrarIncidencia() {
  // Construye la vista de la incidencia en el DOM y vincula botones/eventos
}

function iniciarTemporizador() {
  // Inicia setInterval para el reloj de 25s y setTimeout para tiempo límite
```

```
}

function seleccionarActuacion(indiceOpcion, origenEvento) {
  // Detiene temporizadores, bloquea teclado/ratón, evalúa acierto/fallo y puntúa
}

function gestionarTiempoAgotado() {
  // Procesa la penalización por expiración de tiempo y muestra retroalimentación
}

function registrarResultado(indiceIncidencia, decision, resultado, tiempo, variacion) {
  // Crea e inserta nodo dinámico en el panel de historial
}

function finalizarTurno() {
  // Calcula promedios, genera informe con location.href y actualiza document.title
}

function reiniciarTurno() {
  // Limpia temporizadores y restaura la interfaz a su estado inicial
}
```

Ej3. (60 mins.) Resultados de aprendizaje: 1, 2, 5, 6, 7 y 8

Cree una página HTML para realizar el juego del cuadrado mágico. Un cuadrado mágico es una disposición de números en una cuadrícula de tal forma que la suma de los números en cada fila, cada columna y ambas diagonales principales es la misma.

Cree una tabla con tres filas y tres columnas y permita al usuario introducir números en la misma. Verifique que, al introducir el último número, se cumpla con la propiedad de que todas las filas, columnas y diagonales tengan la misma suma. En caso afirmativo, añada un mensaje de victoria y reinicie el juego. En caso negativo, informe que no se ha llegado a una solución correcta.

Implementa funciones de JavaScript que capturen y procesen los siguientes eventos utilizando el objeto *event*:

- Clic de ratón: Cuando el usuario haga clic en una celda de la tabla, esta se cambiará de color para hacer saber al usuario que esa celda es en la que se escribirá el número que pulse. Al cargar la página web la celda seleccionada será la central.
- Presión de tecla: Cuando el usuario presione una tecla, se deberá comprobar si ésta corresponde a una cifra (números del 0 al 9) o no. En caso negativo, se mostrará un aviso al usuario indicando que sólo se aceptan números. En caso positivo, se colocará el número pulsado en la celda que esté activa en la tabla. Este evento se puede capturar mediante "*keydown*" y se utilizará "*event.keyCode*" o "*event.key*" para identificar la tecla.
- Tecla liberada: Cuando el usuario suelte una tecla (*keyup*) se desencadenará la comprobación del estado del cuadrado mágico en tiempo real. Para ello, se comprobará si el cuadro está completado (hay una cifra en cada celda) y, si es así, si se cumplen las condiciones para que el cuadro se pueda considerar mágico. Si faltan cifras, se marcarán con un color especial los cuadrados no rellenos. Si el cuadrado es mágico, se cambiará el color de todas las celdas del cuadrado a dorado y se mostrará un mensaje dando la enhorabuena al usuario.

Requerirá, al menos, de las siguientes funciones:

```
function seleccionarCelda(event) {
    // Cambiar color de celda activa
    // Actualizar variable de celda seleccionada
}

function validarTecla(event) {
    // Verificar si es número (0-9)
    // Mostrar aviso si no es válido
    // Colocar número en celda activa
}

function verificarCuadradoMagico() {
    // Comprobar si está completo
    // Calcular sumas de filas, columnas y diagonales
    // Verificar si todas las sumas son iguales
    // Mostrar resultado (victoria o continuar)
```

```
}

function reiniciarJuego() {
    // Limpiar todas las celdas
    // Restablecer colores
    // Seleccionar celda central
}
```

Ejemplo de cuadrado mágico válido. La suma mágica es 15 (todas las filas, columnas y diagonales suman 15):

2	7	6
9	5	1
4	3	8

Estados visuales del juego:

1. Celda seleccionada: Fondo azul claro.
2. Celdas vacías: Fondo blanco con borde punteado rojo.
3. Celdas completadas: Fondo gris claro.
4. Cuadrado mágico resuelto: Todas las celdas con fondo dorado.
5. Cuadrado incompleto: Celdas vacías resaltadas en rojo.

Validaciones y retroalimentación:

- Solo acepta números del 1 al 9.
- No permite números repetidos en el cuadrado.
- Muestra el contador de celdas restantes.
- Indica la suma objetivo en tiempo real.
- Proporciona pistas cuando hay errores.

Eventos requeridos:

- *click* - Para seleccionar celdas.
- *keydown* - Para capturar entrada de números.
- *keyup* - Para verificar el estado del cuadrado.
- *load* - Para inicializar el juego.

Funcionalidades adicionales:

- Pista automática: Cada vez que el usuario rellene las 9 celdas y la solución no sea válida, se sumará un intento fallido. Tras 3 intentos fallidos, la aplicación proporcionará una pista colocando automáticamente en la celda correspondiente uno de los números de la solución.
- Cronómetro: Mostrar el tiempo transcurrido desde el inicio.
- Contador de intentos: Contador de número de cuadrados intentados.
- Botón "Nueva partida": Elemento que sirve para reiniciar en cualquier momento.
- Botón "Resolver": Muestra una solución válida.

Mensajes de los posibles resultados:

```
Juego terminado:
        "¡FELICIDADES!
        Has resuelto el cuadrado mágico en [tiempo] segundos
        Suma mágica: [número]
        ¿Quieres intentar otro?"

En caso de error:
        "Este no es un cuadrado mágico
        Las sumas no coinciden. ¡Sigue intentando!"
```

```
En caso de números repetidos:
        "No puedes repetir números
        Cada cifra del 1 al 9 debe aparecer solo una vez"
```

Ej4. (90 mins.) Resultados de aprendizaje: 1, 4, 5, 6, 7 y 8

Cree un simulador de ecosistema virtual donde diferentes especies virtuales interactúan en tiempo real. El usuario podrá añadir, eliminar y observar el comportamiento de las especies mientras estas evolucionan automáticamente. Se utilizará el DOM para crear y manipular elementos dinámicamente, representando un ecosistema que cambia constantemente.

Especies disponibles:

- Herbívoros: Se reproducen rápidamente, comen plantas.
- Carnívoros: Cazan herbívoros, se reproducen lentamente.
- Plantas: Crecen automáticamente, son comida para herbívoros.
- Omnívoros: Comen tanto plantas como herbívoros.

Mecánicas del ecosistema:

- Comportamientos automáticos:
    - Movimiento: Todos los organismos se mueven aleatoriamente cada 2 segundos.
    - Alimentación: Carnívoros buscan herbívoros cercanos, herbívoros buscan plantas.
    - Reproducción: Cuando dos organismos de la misma especie están cerca, pueden reproducirse.
    - Muerte: Los organismos mueren por edad, hambre o por ser cazados.
    - Crecimiento: Las plantas aparecen espontáneamente en posiciones vacías.
- Interacciones del usuario:
    - Clic en organismo: Muestra información detallada en un *tooltip*.
    - Doble clic: Elimina el organismo seleccionado.
    - Arrastrar y soltar: Permite mover organismos manualmente.
    - Botones de control: Pausa/reanuda la simulación.
- Eventos DOM requeridos:
    - click - Para añadir especies y seleccionar organismos.
    - dblclick - Para eliminar organismos específicos.
    - mouseover/mouseout - Para mostrar información de organismos.
    - dragstart/dragend - Para mover organismos manualmente.
    - DOMContentLoaded - Para inicializar el ecosistema.

Propiedades de cada organismo que estarán almacenadas:

```
// Ejemplo de herbívoro
elemento.dataset.tipo = 'herbivoro';
elemento.dataset.energia = '100';
elemento.dataset.edad = '0';
elemento.dataset.velocidad = '2';
elemento.dataset.id = generarIdUnico();

// Ejemplo de planta
elemento.dataset.tipo = 'planta';
elemento.dataset.energia = '40';
elemento.dataset.edad = '0';
elemento.dataset.tamano = '1';
elemento.dataset.rebrota = 'true';
elemento.dataset.id = generarIdUnico();
```

Ejemplo de creación de un organismo:

```
function crearHerbivoro(x, y) {
    const herbivoro = document.createElement('div');
    herbivoro.className = 'organismo herbivoro';

    herbivoro.style.position = 'absolute';
    herbivoro.style.left = x + 'px';
```

```
            herbivoro.style.top = y + 'px';
            herbivoro.style.fontSize = '20px';
            herbivoro.style.cursor = 'pointer';

            // Propiedades específicas
            herbivoro.dataset.tipo = 'herbivoro';
            herbivoro.dataset.energia = '80';
            herbivoro.dataset.edad = '0';

            // Event listeners
            herbivoro.addEventListener('click', mostrarInfoOrganismo);
            herbivoro.addEventListener('dblclick', eliminarOrganismo);

            document.getElementById('ecosistema').appendChild(herbivoro);
            return herbivoro;
        }
```

Posibles estados del ecosistema:

- Equilibrado: Buena proporción entre especies.
- Sobrepoblación: Una especie domina en un ratio de 1.5.
- Extinción: Una especie desaparece completamente.
- Colapso: Muy pocos organismos para mantener el ecosistema.

Interfaz HTML propuesta:

```
<div id="ecosistema-container">
    <div id="panel-control">
        <h1>Control del Ecosistema</h1>
        <button onclick="añadirEspecie('herbivoro')">Añadir Herbívoro</button>
        <button onclick="añadirEspecie('carnivoro')">Añadir Carnívoro</button>
        <button onclick="añadirEspecie('planta')">Añadir Planta </button>
        <button onclick="añadirEspecie('omnivoro')">Añadir Omnívoro</button>
    </div>

    <div id="estadisticas">
        <div>Herbívoros: <span id="count-herbivoro">0</span></div>
        <div>Carnívoros: <span id="count-carnivoro">0</span></div>
        <div>Plantas: <span id="count-planta">0</span></div>
        <div>Omnívoros: <span id="count-omnivoro">0</span></div>
        <div>Generación: <span id="generacion">1</span></div>
    </div>

    <div id="ecosistema">
        <!-- Aquí aparecerán dinámicamente los organismos -->
    </div>

    <div id="log-eventos">
        <h>Registro de Eventos:</p>
        <div id="eventos-lista"></div>
    </div>
</div>
```

Funciones requeridas:

```
        function iniciarEcosistema() {
            // Inicializa el ecosistema con algunos organismos base
            // Establece el ciclo de vida automático con setInterval
            // Crea elementos DOM iniciales
        }

        function añadirEspecie(tipo) {
            // Crea dinámicamente un nuevo elemento DOM para el organismo
            // Asigna posición aleatoria en el ecosistema
            // Establece propiedades específicas según el tipo
            // Añade event listeners para interacción
        }

        function crearOrganismo(tipo, x, y) {
            // Utiliza document.createElement() para crear el organismo
```

```
    // Establece clases CSS, posición y propiedades
    // Añade el elemento al DOM del ecosistema
    // Retorna referencia al elemento creado
}

function simularCicloVida() {
    // Ejecuta cada 2-3 segundos con setInterval
    // Hace que los organismos se muevan, coman, se reproduzcan
    // Elimina organismos que mueren
    // Actualiza estadísticas en tiempo real
}

function moverOrganismo(elemento) {
    // Cambia la posición del elemento en el DOM
    // Utiliza element.style.left y element.style.top
    // Añade animaciones CSS para movimiento suave
}

function interaccionOrganismos() {
    // Detecta cuando dos organismos están cerca
    // Ejecuta comportamientos (comer, reproducirse, huir)
    // Crea/elimina elementos DOM según las interacciones
}

function reproducirEspecie(padre1, padre2) {
    // Crea un nuevo organismo hijo
    // Posiciona cerca de los padres
    // Incrementa la generación
    // Registra evento en el log
}

function eliminarOrganismo(elemento) {
    // Remueve el elemento del DOM con removeChild()
    // Actualiza contadores de especies
    // Añade animación de "muerte" antes de eliminar
}

function actualizarEstadisticas() {
    // Cuenta organismos actuales en el DOM
    // Actualiza los elementos span con los nuevos valores
    // Calcula balances del ecosistema
}

function registrarEvento(mensaje) {
    // Crea elemento de texto dinámicamente
    // Añade timestamp y descripción del evento
    // Inserta en la lista de eventos con insertBefore()
    // Limita a los últimos 10 eventos
}
```

Ampliación de Bibliografía

1. Book of JavaScript, 2nd Edition : A Practical Guide to Interactive Web Pages. Dave Thau. No Starch Press, 2006. https://ebookcentral.proquest.com/lib/bupo-ebooks/detail.action?docID=273476
2. JavaScript Programmer's Reference. Alexei White. John Wiley & Sons, Incorporated, 2009. https://ebookcentral.proquest.com/lib/bupo-ebooks/detail.action?docID=456275

Datos de la Práctica

Estimación temporal:

- Parte presencial: 120 minutos.
    - Explicación inicial: 5 minutos.
    - Experimentos: 60 minutos.
    - Ejercicios: 55 minutos.
- Parte no presencial: 270 minutos.
    - Lectura y estudio del guión y bibliografía básica: 10 minutos
    - Problemas: 260 minutos